

Chapitre 3 : Calcul numérique avec NumPy

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Fil rouge** : les prix des denrées sur les marchés du Sénégal · **Version** : 2026-10-23

Fiche du chapitre

Objectifs (OS2)	A la fin du chapitre, l'apprenant sait : 1) créer un tableau NumPy et lire sa forme et son type ; 2) calculer sur tout un tableau d'un coup (**vectorisation**), et mesurer le gain de vitesse ; 3) calculer des statistiques (moyenne, médiane, écart-type, quantiles) ; 4) sélectionner des valeurs avec des **masques** booléens ; 5) manipuler un tableau à deux dimensions (indexation, tranches, **axes**) ; 6) utiliser le **broadcasting** ; 7) traiter les valeurs manquantes (`nan`).
Prérequis	Chapitres 1 et 2
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (atelier compris)
Matériel	Google Colab ; le fichier des relevés du PAM (fourni)
Livrable	Le notebook de l'atelier 3 : boucle Python contre calcul vectorisé, et l'évolution du prix du mil dans les 14 régions

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Ouvrez un nouveau notebook Colab, `chapitre3-numpy`, envoyez-y le fichier des relevés du PAM, et tapez les exemples dans l'ordre.

---

1. Pourquoi NumPy ?

Au chapitre 2, vous avez calculé des moyennes avec des listes et des boucles. Ça marche, mais c'est **lent** dès que les données grossissent, et c'est long à écrire : une boucle pour convertir, une boucle pour additionner, une autre pour comparer. **NumPy** (*Numerical Python*) fournit un nouveau type, le **tableau** (`ndarray`), qui contient des nombres **tous du même type**, rangés côte à côte en mémoire, et sur lequel on calcule **d'un seul coup**. C'est la base de pandas (chapitre 4), de Matplotlib (chapitre 6) et de presque toutes les bibliothèques de science des données.

2. Le tableau NumPy

```
import numpy as np
print(np.__version__)
```

```
2.5.4
```

`np` est l'abréviation que tout le monde utilise. (Dans Colab, la version peut être différente : les exemples de ce chapitre fonctionnent avec NumPy 2.)

```
prix = np.array([310, 325, 300, 350, 410])
print(prix)
print(type(prix), prix.dtype, prix.shape, prix.ndim)
```

```
[310 325 300 350 410]
<class 'numpy.ndarray'> int64 (5,) 1
```

Morceau	Ce qu'il veut dire
`np.array([...])`	Crée un tableau à partir d'une liste
`prix.dtype`	Le **type** des éléments : `int64` (entiers), `float64` (décimaux), `bool` (vrai / faux), `<U11` (textes)
`prix.shape`	La **forme** : `(5,)` veut dire « 5 éléments, une seule dimension »
`prix.ndim`	Le nombre de **dimensions** (1 pour une ligne, 2 pour un tableau à lignes et colonnes)

La différence avec une liste saute aux yeux dès qu'on calcule :

```
liste = [310, 325, 300, 350, 410]
print(liste * 2)
print(prix * 2)
```

```
[310, 325, 300, 350, 410, 310, 325, 300, 350, 410]
[620 650 600 700 820]
```

Une liste multipliée par 2 est **répétée** ; un tableau multiplié par 2 est multiplié **élément par élément**.

> 🪛 **Essayez** : `np.array([1, 2, "trois"])`. Le tableau est créé, mais son type est `<U21` : **tout** est devenu du texte, et `* 2` échouera. Un tableau ne contient qu'un seul type ; NumPy choisit celui qui peut tout contenir. Annulez.

3. Calculer sur tout le tableau

```
print(prix + 15)
print(prix / 655.957)
print(prix - prix.mean())
```

```
[325 340 315 365 425]
[0.47259195 0.49545931 0.45734705 0.53357156 0.62504097]
[-29. -14. -39.  11.  71.]
```

Morceau	Ce qu'il veut dire
`prix + 15`	15 F de plus sur **chaque** prix
`prix / 655.957`	Les prix en **euros** : la parité entre le franc CFA et l'euro est fixe, 1 euro = 655,957 F CFA
`prix - prix.mean()`	L'écart de chaque prix à la moyenne (339 F)

Aucune boucle : c'est la **vectorisation**. On écrit l'opération une fois, NumPy l'applique à tous les éléments.

4. Les prix du riz importé

Chargeons les 9 308 prix au kilo du riz importé, avec le module `csv` du chapitre 2, puis passons-les à NumPy :

```
import csv

riz = []
with open("wfp_food_prices_sen.csv", newline="", encoding="utf-8") as fichier:
    for r in csv.DictReader(fichier):
        if r["commodity"] == "Rice (imported)" and r["unit"] == "KG":
            riz.append(float(r["price"]))
prix_riz = np.array(riz)
print(prix_riz.shape, prix_riz.dtype)
print(prix_riz[:5])
```

```
(9308,) float64
[227.5  226.88 227.5  228.33 227.5 ]
```

`prix_riz[:5]` : les tranches fonctionnent comme avec les listes.

```
print(f"moyenne {prix_riz.mean():.1f}, médiane {np.median(prix_riz):.1f}")
print(f"écart-type {prix_riz.std():.1f}, min {prix_riz.min()}, max {prix_riz.max()}")
print("quartiles :", np.percentile(prix_riz, [25, 50, 75]))
```

```
moyenne 310.7, médiane 300.0
écart-type 55.7, min 150.0, max 600.0
quartiles : [275.   300.   343.75]
```

Morceau	Ce qu'il veut dire
`.mean()`, `.min()`, `.max()`	Moyenne, minimum, maximum
`np.median(...)`	La **médiane** : la moitié des prix sont en dessous, la moitié au-dessus
`.std()`	L'**écart-type** : de combien les prix s'écartent de la moyenne, en moyenne (55,7 F)
`np.percentile(..., [25, 50, 75])`	Les **quartiles** : 25 % des prix sont sous 275 F, 75 % sous 343,75 F

5. Sélectionner avec des masques

Une **comparaison** sur un tableau donne un tableau de booléens, appelé **masque** :

```
masque = prix_riz > 400
print(masque[:5], masque.dtype)
print(masque.sum(), "relevés au-dessus de 400 F, soit", f"{masque.mean():.1%}")
print(prix_riz[masque][:5])
```

```
[False False False False False] bool
827 relevés au-dessus de 400 F, soit 8.9%
[475.   467.   437.5  500.   456.25]
```

Morceau	Ce qu'il veut dire
`prix_riz > 400`	Pour chaque prix, `True` s'il dépasse 400 F
`masque.sum()`	`True` compte pour 1 : le **nombre** de prix au-dessus de 400 F
`masque.mean()`	La **proportion**
`prix_riz[masque]`	Seulement les prix où le masque est `True`

On combine plusieurs conditions avec `&` (et), `|` (ou), `~` (non), **en mettant chaque condition entre parenthèses** :

```
entre = (prix_riz >= 300) & (prix_riz <= 400)
print(entre.sum(), "relevés entre 300 et 400 F")
print(np.where(prix_riz > 550, "très cher", "normal")[:3])
```

```
4564 relevés entre 300 et 400 F
['normal' 'normal' 'normal']
```

`np.where(condition, si_vrai, si_faux)` construit un nouveau tableau, élément par élément.

> 🖍️ **Essayez** : écrivez `prix_riz >= 300 and prix_riz <= 400`. Message : `ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`. Le `and` de Python veut **un seul** vrai ou faux ; pour un tableau, il faut `&`. Et sans les parenthèses (`prix_riz >= 300 & prix_riz <= 400`), `&` est calculé **avant** les comparaisons, sur `300 & prix_riz` : `TypeError: ufunc 'bitwise_and' not supported for the input types...`. Annulez.

6. La vitesse

Comparons une boucle et NumPy pour convertir un million de prix en euros et en faire la somme :

```python
import time

grand = np.random.default_rng(0).uniform(100, 1000, size=1_000_000)
liste_grand = grand.tolist()

debut = time.perf_counter()
total_boucle = 0
for p in liste_grand:
    total_boucle += p / 655.957
duree_boucle = time.perf_counter() - debut

debut = time.perf_counter()
total_numpy = (grand / 655.957).sum()
duree_numpy = time.perf_counter() - debut

print(f"boucle : {duree_boucle * 1000:.0f} ms ; NumPy : {duree_numpy * 1000:.1f} ms ; rapport
        {duree_boucle / duree_numpy:.0f}")
print(round(total_boucle, 2) == round(total_numpy, 2))
```

```
boucle : 137 ms ; NumPy : 8.3 ms ; rapport 17
True
```

(Les durées dépendent de l'ordinateur et changent à chaque exécution.)

Morceau	Ce qu'il veut dire
`np.random.default_rng(0)`	Un générateur de nombres au hasard ; la **graine** 0 donne toujours les mêmes nombres
`.uniform(100, 1000, size=1_000_000)`	Un million de prix tirés au hasard entre 100 et 1 000
`.tolist()`	Le même contenu, en liste Python (pour une comparaison juste)

Même résultat, une quinzaine de fois plus vite, en une ligne au lieu de quatre. Sur des opérations plus lourdes, le rapport monte à 50 ou 100. NumPy est rapide parce que la boucle existe toujours, mais elle est écrite en langage C, sur des nombres rangés côte à côte, au lieu d'être interprétée ligne par ligne par Python.

7. Un tableau à deux dimensions

Construisons un tableau à **deux dimensions** : le prix moyen de 4 céréales (les lignes) pendant 5 années (les colonnes).

```
denrees = ["Millet", "Rice (imported)", "Maize (local)", "Sorghum"]
annees = [2021, 2022, 2023, 2024, 2025]
sommes = np.zeros((len(denrees), len(annees)))
comptes = np.zeros((len(denrees), len(annees)))
with open("wfp_food_prices_sen.csv", newline="", encoding="utf-8") as fichier:
    for r in csv.DictReader(fichier):
        if r["commodity"] in denrees and r["unit"] == "KG" and int(r["date"][:4]) in annees:
            i = denrees.index(r["commodity"])
            j = annees.index(int(r["date"][:4]))
            sommes[i, j] += float(r["price"])
            comptes[i, j] += 1
moyennes = sommes / comptes
print(moyennes.shape)
print(moyennes.round(0))
```

```
(4, 5)
[[260. 395. 424. 334. 304.]
 [321. 355. 352. 426. 384.]
 [239. 354. 350. 302. 286.]
 [259. 398. 413. 381. 363.]]
```

Morceau	Ce qu'il veut dire
`np.zeros((4, 5))`	Un tableau de 4 lignes et 5 colonnes rempli de 0
`denrees.index(...)`	La position de la denrée dans la liste : le **numéro de ligne**
`sommes[i, j] += ...`	La case de la ligne `i`, colonne `j`
`sommes / comptes`	La division case par case : 20 moyennes d'un coup

Lisez le tableau : le **mil** (première ligne) passe de 260 F en 2021 à 424 F en 2023, alors que le **riz importé** (deuxième ligne) reste stable jusqu'en 2023. Les céréales produites au Sénégal ont flambé en 2022-2023 ; le riz importé, lui, a monté plus tard, en 2024.

7.1 Indexer et découper

```
print(moyennes[1])
print(moyennes[:, -1])
print(moyennes[1, -1])
print(moyennes[0:2, 3:])
```

```
[321.41444444 355.42459016 351.7309322  426.12602459 384.44866348]
[303.9837819  384.44866348 286.40685366 363.00059406]
384.4486634844869
[[333.50515464 303.9837819 ]
 [426.12602459 384.44866348]]
```

Morceau	Ce qu'il veut dire
`moyennes[1]`	La ligne 1 : le riz importé, les 5 années
`moyennes[:, -1]`	**Toutes** les lignes (`:`), dernière colonne (`-1`) : les 4 céréales en 2025
`moyennes[1, -1]`	Une seule case : le riz importé en 2025
`moyennes[0:2, 3:]`	Les lignes 0 et 1, les colonnes à partir de la 3 (2024 et 2025)

> 🧨 **Essayez** : `moyennes[4, 0]` . Message : `IndexError: index 4 is out of bounds for axis 0 with size 4` (« l'indice 4 dépasse l'axe 0, qui a 4 éléments ») : les lignes sont numérotées de 0 à 3. Annulez.

8. Les axes

Une statistique sur un tableau à deux dimensions peut se calculer **par ligne** ou **par colonne**. C'est le paramètre `axis` :

```
print("par denrée (moyenne des 5 ans) :", moyennes.mean(axis=1).round(0))
print("par année (moyenne des 4 denrées) :", moyennes.mean(axis=0).round(0))
```

```
par denrée (moyenne des 5 ans) : [343. 368. 306. 363.]
par année (moyenne des 4 denrées) : [270. 376. 385. 361. 334.]
```

Paramètre	Ce qu'il fait	Résultat
`axis=0`	Écrase les **lignes** : un résultat par **colonne**	5 valeurs, une par année
`axis=1`	Écrase les **colonnes** : un résultat par **ligne**	4 valeurs, une par denrée
sans `axis`	Tout le tableau	1 seule valeur

> 🟢 **Débutant : un moyen de s'en souvenir** L'axe qu'on donne est celui qui **disparaît**. Le tableau a la forme `(4, 5)` ; avec `axis=0`, la dimension 4 disparaît, il reste 5 valeurs. Avec `axis=1`, la dimension 5 disparaît, il reste 4 valeurs.

9. Le broadcasting

Pour comparer des denrées de prix différents, on calcule un **indice base 100** : chaque prix divisé par celui de 2021, fois 100. Il faut diviser chaque **ligne** par **son** prix de 2021.

```
base_2021 = moyennes[:, 0]
print(base_2021.shape, moyennes.shape)
indice = moyennes / base_2021[:, np.newaxis] * 100
print(indice.round(0))
```

```
(4,) (4, 5)
[[100. 152. 163. 128. 117.]
 [100. 111. 109. 133. 120.]
 [100. 148. 147. 127. 120.]
 [100. 154. 159. 147. 140.]]
```

Morceau	Ce qu'il veut dire
`base_2021 = moyennes[:, 0]`	La première colonne : les 4 prix de 2021, forme `(4,)`
`base_2021[:, np.newaxis]`	La même chose « en colonne » : forme `(4, 1)`
`moyennes / ...`	NumPy **étire** la colonne `(4, 1)` sur les 5 colonnes de `(4, 5)` : c'est le **broadcasting** (« diffusion »)

Le sorgho coûte 1,54 fois plus cher en 2022 qu'en 2021 (indice 154), et 1,40 fois plus en 2025.

Sans `np.newaxis`, NumPy essaie d'aligner les formes par la **droite** : `(4,)` contre `(4, 5)`, il compare 4 et 5, et refuse :

```
try:
    moyennes / base_2021
except ValueError as e:
    print("Erreur :", e)
```

```
Erreur : operands could not be broadcast together with shapes (4,5) (4,)
```

🔵 **Standard : la règle du broadcasting** NumPy compare les formes **en partant de la droite**. Deux dimensions sont compatibles si elles sont **égales**, ou si l'une vaut **1** (elle est alors étirée). `(4, 5)` et `(4, 1)` : 5 et 1 compatibles, 4 et 4 égales. `(4, 5)` et `(5,)` : 5 et 5 égales, compatible aussi (chaque ligne est divisée par le même vecteur de 5 valeurs). `(4, 5)` et `(4,)` : 5 et 4, incompatibles.

10. Les valeurs manquantes : `nan`

NumPy note une valeur manquante `nan` (*not a number*). Elle « contamine » les calculs :

```
valeurs = np.array([310.0, np.nan, 300.0, 350.0])
print(valeurs.mean(), np.nanmean(valeurs))
print(np.isnan(valeurs))
```

```
nan 320.0
[False  True False False]
```

Morceau	Ce qu'il veut dire
`np.nan`	Une valeur manquante (de type `float`)
`valeurs.mean()`	`nan` : une seule valeur manquante suffit à rendre la moyenne inconnue
`np.nanmean(...)`	La moyenne **en ignorant** les `nan` ; existent aussi `np.nansum`, `np.nanmax`…
`np.isnan(...)`	Le masque des valeurs manquantes (`valeurs == np.nan` ne marche **pas** : `nan` n'est égal à rien, pas même à lui-même)

🟢 **Débutant : copie ou même tableau ?** `b = a` ne copie pas un tableau : `a` et `b` désignent le **même** tableau, et `b[0] = 99` modifie aussi `a`. Pour une vraie copie : `b = a.copy()`. C'est la même chose qu'avec les listes, mais avec de gros tableaux, l'erreur passe facilement inaperçue.

Exercices

Dans votre notebook du chapitre, à la suite des exemples :

- ★ **Exercice 1 : les températures.** Créez un tableau des températures moyennes mensuelles (valeurs **fictives**, proches de celles de Dakar) : 21,5 ; 21,0 ; 21,8 ; 22,6 ; 24,0 ; 26,5 ; 27,6 ; 27,8 ; 28,0 ; 27,9 ; 26,1 ; 23,4. Convertissez-les en degrés Fahrenheit (°C x 9 / 5 + 32), affichez les mois au-dessus de 27 °C (avec un tableau des noms de mois et un masque), la moyenne annuelle et l'écart entre le mois le plus chaud et le plus frais. Attendu : juillet, août, septembre, octobre ; 24,85 °C ; 7 °C. (Sections 3 et 5.)
- ★ **Exercice 2 : le riz bon marché.** Sur `prix_riz` : combien de relevés sont sous 250 F, et quelle part cela représente-t-il ? Quelle est la médiane des relevés au-dessus de 400 F ? Attendu : 621 relevés, 6,7 % ; 433,33 F. (Sections 4 et 5.)

- ★★ **Exercice 3 : remodeler.** `np.arange(1, 37) * 10` donne 36 ventes mensuelles (de 10 à 360) sur 3 ans. Remodelez-les en un tableau de 3 lignes (les années) et 12 colonnes (les mois) avec `.reshape(3, 12)`. Calculez le total de chaque année, les ventes de décembre de chaque année, et le total de janvier, février et mars sur les 3 ans. (Sections 7 et 8.)
- ★★ **Exercice 4 : la variation d'une année sur l'autre.** A partir du tableau `moyennes` de la section 7, calculez la variation en pourcentage d'une année à la suivante pour chaque denrée, avec `np.diff(moyennes, axis=1)` (les différences entre colonnes voisines) divisé par `moyennes[:, :-1]`. Quelle denrée a la plus forte hausse en une année ? (Sections 7 à 9.)
- ★★★ **Exercice 5 : l'année la plus chère.** Pour chaque denrée du tableau `moyennes`, en quelle année le prix moyen a-t-il été le plus élevé (`argmax` avec `axis=1`) ? Puis, sur tout le tableau, quelle case a le prix le plus élevé ? Indice : `np.unravel_index(moyennes.argmax(), moyennes.shape)` transforme la position « à plat » en (ligne, colonne). (Sections 7 et 8.)

Atelier 3 : boucle Python contre calcul vectorisé

Le travail principal du chapitre : vous chargez les 51 854 prix au kilo dans des tableaux NumPy, vous les convertissez en euros, vous sélectionnez avec des masques, vous comparez la vitesse d'une boucle et du calcul vectorisé sur un vrai calcul statistique, puis vous construisez le tableau du prix du mil dans les 14 régions, de 2015 à 2025, et l'indice base 100 qui montre où il a le plus augmenté. L'énoncé et le notebook de départ vous sont fournis séparément.

A retenir

- `np.array(liste)` ; `.shape` (forme), `.dtype` (type, un seul pour tout le tableau), `.ndim`.
- Les opérations s'appliquent **élément par élément**, sans boucle (vectorisation) : plus court et beaucoup plus rapide.
- Statistiques : `.mean()`, `np.median`, `.std()`, `.min()`, `.max()`, `np.percentile`.
- Masque : `t > 400` donne des booléens ; `t[masque]` sélectionne ; `.sum()` compte ; `&`, `|`, `~` avec des parenthèses ; jamais `and` / `or`.
- Deux dimensions : `t[i, j]`, `t[:, j]` (une colonne), `t[i]` (une ligne) ; `axis=0` donne un résultat par colonne, `axis=1` par ligne.
- Broadcasting : les formes se comparent par la droite ; une dimension 1 est étirée ; `[:, np.newaxis]` pour une colonne.
- `np.nan` contamine les calculs : `np.nanmean`, `np.isnan`.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`ValueError: The truth value of an array with more than one element is ambiguous`	`and`, `or` ou `if` sur un tableau entier	`&`, `|` avec parenthèses ; `masque.any()` ou `masque.all()` dans un `if`

Message ou symptôme	Ce que ça veut dire	Solution
`ValueError: operands could not be broadcast together with shapes (4,5) (4,)`	Formes incompatibles	`[:, np.newaxis]` pour diviser ligne par ligne ; vérifier les `.shape`
`IndexError: index 4 is out of bounds for axis 0 with size 4`	Indice trop grand	Les indices vont de 0 à taille - 1
Le tableau est de type `<U...` et les calculs échouent	Un texte dans les données : tout est devenu texte	Convertir avant (`float`), ou nettoyer
Une moyenne vaut `nan`	Une valeur manquante dans le tableau	`np.nanmean` ; chercher les `nan` avec `np.isnan`
`RuntimeWarning: Mean of empty slice`	Moyenne d'un tableau vide (un masque qui ne sélectionne rien)	Vérifier `masque.sum()` avant
`RuntimeWarning: invalid value encountered in divide`	Division 0 / 0 (une case sans relevé)	`np.divide(a, b, out=..., where=b > 0)` (atelier)
Un tableau change « tout seul »	`b = a` au lieu de `b = a.copy()`	`.copy()`

Pour aller plus loin

- NumPy pour les grands débutants, guide officiel (en anglais) : https://numpy.org/doc/stable/user/absolute_beginners.html
- Le broadcasting expliqué avec des dessins (en anglais) : https://numpy.org/doc/stable/user/basics.broadcasting.html
- *Python Data Science Handbook*, de Jake VanderPlas, chapitre 2 sur NumPy, en accès libre (en anglais) : https://jakevdp.github.io/PythonDataScienceHandbook/